

NumStability Reuse Across 38 HighamBench Tasks

A category-free analysis of accepted Lean proofs and a comparison with the ten-task formalize-and-prove benchmark

29 September 2026 | 31 paper identifiers | 114 paired repetitions | 228 accepted attempts

This is a *new presentation of existing data*, not a new model run. The earlier usage report and its figures remain intact. All 38 eligible tasks are included. Historical suffixes in task IDs are retained for traceability, but no expected-benefit category is used to group results, color figures, or calculate associations.

Abstract

We ask whether an AI prover’s *observed use* of NumStability is associated with reduced effort on fixed paper-derived Lean proof tasks. Condition N saw Lean and Mathlib, whereas condition L also saw a frozen NumStability library. Every selected task has three accepted N/L proof pairs. In 14 tasks all three accepted L proofs depended on NumStability; their task-mean gains averaged +54.0% elapsed time and +41.7% submitted source lines. In 18 tasks none did; corresponding averages were +5.6% time and −2.2% lines. L was faster in 29/38 tasks and submitted fewer source lines in 22/38. These are descriptive *post-run associations*, not causal estimates: use is a model decision, source-campaign and task characteristics are strongly confounded, and historical task labels were originally chosen using expected library benefit. This report keeps both favorable and unfavorable outcomes, including six mixed-use tasks, and compares the study with the distinct ten-task formalize-and-prove benchmark.¹

1 Question, scope, and source campaigns

The narrow question answered here is: *when the final accepted proof actually reaches NumStability declarations, how do N/L time, observed tokens, and submitted code size compare?* This does not measure how much potentially useful library mathematics remained undiscovered. Nor does an audited dependency alone prove that the library caused a gain. The archived task IDs are preserved exactly; their final suffixes are identifiers, not analysis strata.

The earlier selected-tier results drew on two sources. The later **results-r11** campaign supplied 15 tasks (45 accepted pairs); the combined **results-r09/results-r10** evidence supplied 23 further tasks (69 accepted pairs). **r10** was a fresh replacement campaign for infrastructure-affected **r09** pairs, not an extra repetition. Each of these 38 tasks has three complete, accepted N/L repetitions. The false-target P16-T3 failure was excluded from the original complete-pair summary; it is not silently classified as an ordinary proof failure here. The two historical P12-T1 and P17-T1 results concern earlier target versions, not their later redesigned tasks.²

The task selection originally used expected library benefit. This report intentionally does not use those expectations as an outcome variable, but their role in construction remains a source of selection bias. Some tasks may later be judged mathematically incompatible or outside the intended scope. For this descriptive report, no additional result-aware removals have been made; future eligibility decisions should state a source-based rule before recomputing a primary estimate.

¹Primary task ledger: `../library-usage-analysis/task_usage.csv`; pair ledger: `../library-usage-analysis/pair_usage.csv`; independent ten-task report: NumStability repository, `Documentation/numstability_ten_task_report.pdf`, SHA-256 8b2cd10787487cb4....

²Source summaries: `../live-r11/README.md` and `../combined-r09-r10/README.md`. Run history and target caveats: `../MEASUREMENT_PROGRESS.md`.

2 What the larger benchmark actually measured

Conditions and work unit

The larger study was a *fixed-target proof benchmark*. Both conditions received the same paper-scoped task context and an identical, already chosen Lean theorem signature. The agent had to preserve its namespace, name, arguments, assumptions, and conclusion, and write a complete `Candidate.lean` proof. N had Mathlib but no NumStability source, compiled objects, or index. L had the same environment plus read-only NumStability source and compiled modules, with a short access guide encouraging local search and imports. There was no one-time orientation scout. Final proof validation rejected `sorry`, `admit`, extra axioms, unsafe trust escapes, forbidden imports, and edits to the target. One fresh agent/workspace state was used per condition attempt, with no cross-task solution sharing.³

The measured clock began at authenticated prompt release and ended at the authenticated final submission. Hidden final validation ran afterward and was logged separately. Intermediate Lean checks by a contestant counted as contestant work. The cap was 7,200 seconds per condition, without a token ceiling; final recorded token usage is incomplete and represents provider-observed lower bounds. Physical source lines count the submitted file, not imported Mathlib or NumStability implementation lines. A passing fixed-target proof is not itself a fresh paper-faithfulness audit; task-construction/source-review evidence is a separate question.⁴

Historical execution differences that matter

Aspect	r09/r10 source	r11 source
Campaign role	r09 original measurement; r10 fresh replacement of 30 infrastructure-affected pairs	Later 38-task campaign; this report takes its 15 completed tasks
Model	GPT-5.6 Sol, xhigh	GPT-5.6 Sol, xhigh
Agent parallelism	Measured-v4 allowed root/child work	Measured-v5 locked one root and disabled subagents
Host execution	Four disjoint CPU slots; N/L attempts could run concurrently	Four disjoint CPU slots; concurrent measured attempts also shared provider authentication
Per-attempt limit	7,200 s, no token cap; r09/r10 used four-core, 16-GiB slots	7,200 s, no token cap; four-slot CPU lease
Interpretation	Pair replacements selected by infrastructure rule	Later protocol version; not a controlled rerun of r09/r10

The run journal later identified shared-provider queue/cache interaction and a changed model-visible host developer context between campaigns. A subsequent measured-v6 protocol serialized provider-active attempts precisely because older campaigns could not be pooled as a clean common-timing experiment. This report therefore shows all archived outcomes and their relationships but does *not* claim that the combined 38-task mean isolates a single treatment effect.⁵

³Tracked prompt: `../../measurement_prompt.md`; L supplement: `../../condition_prompts/L.md`; task/run history: `../../MEASUREMENT_PROGRESS.md`. The current tracked protocol JSON is measured-v6 and must *not* be mistaken for the earlier r09/r10 or r11 runtime lock.

⁴Timing and LOC definitions: `../live-r11/README.md`, `../combined-r09-r10/README.md`, and the accepted attempt records indexed in `../library-usage-analysis/attempt_usage.csv`.

⁵`../../MEASUREMENT_PROGRESS.md`, especially the measured-v4, measured-v5, and measured-v6 launch/correction entries.

3 Post-run usage measures and outcome definitions

The primary usage measure is deliberately simple: for each task, count how many of its three accepted L proofs have at least one `NumStability` declaration in the saved, complete hidden Lean dependency audit. This yields an integer from 0 to 3. It is *observed uptake*, not pre-run coverage. An import alone is not counted as use. The saved audit also permits complementary checks: source-name references, direct distance-one dependencies of the accepted target, filtered/public transitive dependencies, and matches against task-listed candidate declarations. These answer different questions; none alone identifies causal reuse or useful unused material. All 114 N accepted proofs have zero audited NumStability dependencies.⁶

For each repetition, gain is $(N - L)/N \times 100\%$, so positive favors L. For the main task table, we first average the three N measurements and the three L measurements, then calculate gain from those task means. Grouped means give each task equal weight. Elapsed time is contestant-active time to an accepted fixed proof; it cannot be split into formalization and proof phases here because the theorem was fixed before the contest. Token differences are shown for transparency, but *all 228 selected token records are incomplete*; neither their sign nor their magnitude is an exact full-spend comparison.

Metric across 38 task means	N mean	L mean	Gain of totals	Tasks L better	Unit	Qualification
Elapsed time	753.1	517.4	+31.3%	29/38	seconds	mixed historical protocols
Observed tokens	2.453M	2.162M	+11.9%	25/38	tokens	incomplete lower bounds
Submitted physical LOC	399.7	340.5	+14.8%	22/38	lines	excludes imported code

The ratio-of-totals above weights long and short tasks by their resource use; the grouped table below instead averages percentage gains equally over tasks. Neither summary should be presented as a randomized effect.

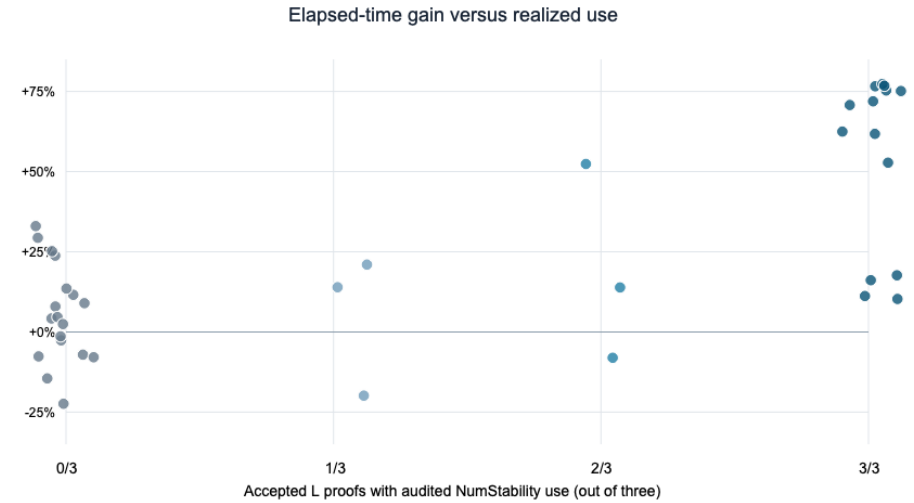
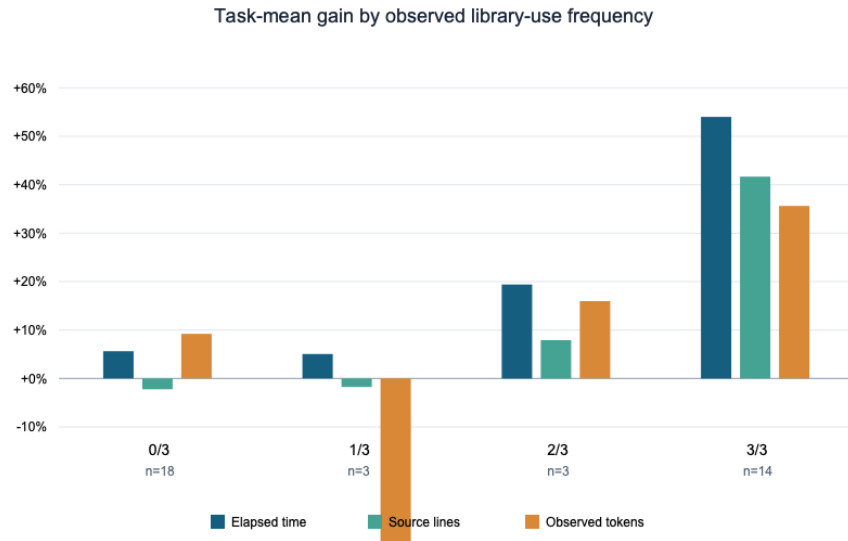
4 Results grouped only by realized uptake

L proofs using library	Tasks	Mean time gain	Mean LOC gain	Mean observed-token gain	L faster	L shorter
0/3	18	+5.6%	-2.2%	+9.2%	11/18	8/18
1/3	3	+5.0%	-1.8%	-84.3%	2/3	1/3
2/3	3	+19.4%	+7.9%	+16.0%	2/3	1/3
3/3	14	+54.0%	+41.7%	+35.6%	14/14	12/14

The all-three-use group is markedly more favorable than the no-use group, but this is a post-treatment grouping. The formalizer’s decision to use the library, original task design, difficulty, and campaign version may all move together. Indeed, almost all all-three-use tasks are from r11, while most zero-use tasks are from r09/r10; the plot is not an independent cross-campaign identification strategy. Within tasks that mix using and non-using repetitions, the pattern is more heterogeneous.⁷

⁶Definitions and raw hashes: `../library-usage-analysis/report.tex`, `../library-usage-analysis/attempt_usage.csv`, `../library-usage-analysis/declaration_usage.csv`.

⁷`../library-usage-analysis/mixed_task_usage.csv` and `../library-usage-analysis/pair_usage.csv`.



Left: equal-weight mean task gains (not uncertainty intervals). Right: every task's time gain, jittered horizontally only to reveal overlapping points.

Campaign provenance is entangled with observed use

Source of selected task means	Tasks	0/3 use	1/3 use	2/3 use	3/3 use
Combined r09/r10	23	18	2	2	1
r11	15	0	1	1	13

The 14 consistently using tasks comprise 13 from r11 and one from combined r09/r10. Conversely, all 18 tasks with no observed use are from r09/r10. The visual gradient therefore spans different task mixes and execution protocols. It is a useful diagnostic description, not a within-protocol effect estimate.

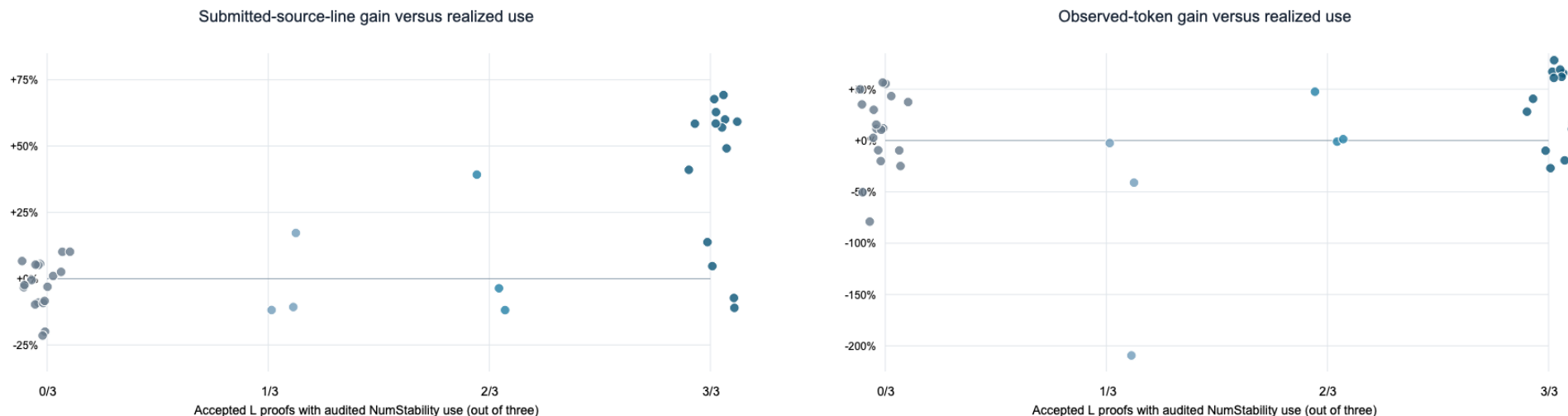
5 The full task ledger

Every selected task appears below, including negative outcomes. “Use” is the number of three L proofs with any audited NumStability dependency; “Hit” is the number intersecting the historical task candidate list. The latter is a secondary, expectation-linked measure, not the basis of grouping. N/L columns are three-run task means. Token gains are shown although their exact totals are incomplete.

Table 1: All 38 accepted tasks; positive gains favor L. IDs remain unchanged for artifact traceability.

Task	Use	Hit	N time	L time	Time gain	N LOC	L LOC	LOC gain	Token gain
P01-T1	2/3	2/3	470	224	+52.4%	138	84	+39.2%	+47.6%
P01-T2	3/3	3/3	841	208	+75.3%	311	124	+60.0%	+65.6%
P02-T1	3/3	3/3	1238	1038	+16.2%	416	397	+4.7%	-26.9%
P02-T3	1/3	0/3	1315	1577	-19.9%	872	965	-10.7%	-209.4%
P03-T3	0/3	0/3	322	369	-14.5%	317	319	-0.5%	-79.0%
P04-T1	3/3	3/3	807	716	+11.2%	430	371	+13.8%	-10.0%
P05-T3	0/3	0/3	975	888	+9.0%	633	569	+10.1%	-24.9%
P06-T3	0/3	0/3	363	321	+11.6%	350	346	+1.0%	+43.3%
P08-T3	0/3	0/3	947	635	+33.0%	677	632	+6.7%	+49.8%
P09-T3	0/3	0/3	4608	4242	+7.9%	2305	2513	-9.0%	+11.4%
P11-T3	0/3	0/3	547	586	-7.1%	353	344	+2.5%	-9.7%
P12-T1	3/3	3/3	623	294	+52.8%	254	129	+49.1%	-19.4%
P13-T3	0/3	0/3	779	550	+29.4%	331	342	-3.2%	+35.1%
P14-T1	3/3	3/3	559	460	+17.7%	234	251	-7.3%	+11.1%
P15-T1	3/3	3/3	694	173	+75.1%	265	108	+59.2%	+55.2%
P15-T2	3/3	3/3	1661	389	+76.6%	727	271	+62.8%	+78.2%
P15-T3	3/3	3/3	899	806	+10.3%	730	811	-11.0%	+17.0%
P17-T1	1/3	0/3	295	233	+21.0%	137	114	+17.2%	-41.0%
P17-T3	0/3	0/3	176	168	+4.2%	137	150	-9.7%	+2.7%
P19-T3	0/3	0/3	433	374	+13.6%	428	441	-3.0%	+55.3%
P20-T3	1/3	1/3	479	412	+14.0%	451	504	-11.8%	-2.5%
P23-T1	3/3	3/3	937	351	+62.5%	439	259	+41.0%	+28.1%
P24-T3	0/3	0/3	196	240	-22.4%	114	136	-19.9%	+11.9%
P25-T3	0/3	0/3	203	208	-2.6%	116	127	-9.2%	+10.5%
P26-T1	3/3	3/3	1423	399	+71.9%	540	174	+67.7%	+67.0%
P27-T3	0/3	0/3	301	287	+4.6%	204	192	+5.6%	-9.5%
P28-T1	3/3	0/3	726	165	+77.3%	260	112	+57.0%	+69.1%
P28-T3	0/3	0/3	225	242	-7.9%	190	171	+10.2%	+37.5%
P29-T1	3/3	0/3	641	149	+76.8%	241	74	+69.3%	+62.0%
P29-T2	3/3	3/3	2092	799	+61.8%	994	412	+58.5%	+61.0%
P32-T3	0/3	0/3	237	255	-7.6%	165	169	-2.4%	-50.4%
P33-T1	3/3	1/3	766	224	+70.8%	273	114	+58.4%	+40.6%
P34-T3	0/3	0/3	248	189	+23.8%	214	203	+5.1%	+15.5%
P35-T3	2/3	0/3	248	268	-8.1%	239	248	-3.6%	-1.1%
P36-T3	0/3	0/3	276	280	-1.4%	120	145	-21.4%	-20.1%
P37-T3	0/3	0/3	453	442	+2.4%	227	246	-8.4%	+56.4%
P39-T3	0/3	0/3	282	211	+25.2%	158	149	+5.3%	+29.9%
P40-T3	2/3	0/3	335	288	+13.9%	200	223	-11.9%	+1.4%

6 Size and token outcomes; within-task diagnostics



Every task is plotted; colors encode only the observed 0–3 uptake count. The token axis shows provider-observed lower-bound differences.

Six tasks have a mixture of using and non-using L repetitions. For P01-T1, the two using repetitions average +76.8% time gain and +64.2% LOC gain, while the non-using repetition shows −12.4% and −24.8%. That is suggestive *within the same target*, but just three repetitions cannot establish a stable effect. Other mixed-use tasks do not show the same strong pattern: P35-T3’s using repetitions are slower on average than its non-using repetition. A dependency may be mathematically nearby yet insufficient to shorten the particular proof.⁸

P15-T3 is a useful exception to any sweeping rule: all three L proofs have audited library dependencies, but L’s mean submitted source is 11.0% longer, despite a 10.3% time gain. Conversely, some no-use tasks favor L, which can arise from model/run variability and is not evidence of a NumStability mechanism. Availability, retrieval, usage, and mathematical leverage are distinct concepts. This dataset directly observes only final proof dependency and submitted output, not every abandoned search or an unused library opportunity.

⁸../library-usage-analysis/mixed_task_usage.csv; accepted proof and audit paths in ../library-usage-analysis/attempt_usage.csv.

7 Comparison with the separate ten-task benchmark

The NumStability Documentation/numstability_ten_task_report.pdf report describes a different question and execution. The following is a *side-by-side protocol comparison*, not a pooled meta-analysis.⁹

Dimension	This 38-task fixed-target analysis	Separate ten-task formalize-and-prove report
Work assigned	Prove an already fixed Lean theorem without changing its signature	Derive a source-faithful Lean statement from a paper, then prove that frozen proposition
Faithfulness gate	Upstream task/source review; no fresh candidate-specific statement audit during a fixed proof attempt	Independent audit for each generated statement, with repairs before proof
Sample	38 tasks from 31 paper IDs; three N/L repetitions per task; two historical protocol generations combined	Ten tasks from five source clusters; one N/L pair per task; outcome-aware chosen corpus
Formalizers	GPT-5.6 Sol/xhigh; old r09/r10 and later r11 controls differ	GPT-6 Sol/high; agents prohibited from spawning subagents
Library familiarization	L-only library access guide; no shared pre-task warm conversation	L-only task-neutral scout once, then task conversations fork from the orientation root
Measured phases	One timed proof task; no formalization/proof time split	Separate timed faithful-formalization and complete-proof stages; one-time scout separately charged
Hardware	Titan, four disjoint four-core slots in r09/r10 and r11; historic memory/control records differ	Titan, three disjoint 8-logical-CPU/24-GiB lanes
Code metric	Full submitted physical source lines for accepted file	Nonblank proof-code lines; statement lines separately reported
Token metric	Incomplete provider-observed totals, lower bounds	Net-new input/output tokens, cached input accounted separately
Headline descriptive outcome	29/38 tasks faster for L; 22/38 shorter; gain of totals +31.3% time and +14.8% physical LOC	8/10 tasks faster for L; 6/10 shorter in proof LOC; +22.5% active time, +19.7% proof LOC, −34.7% net-new tokens

The smaller study reports faithful statements and zero-hole proofs on all ten pairs, and its L task-active formalization was 7.3% slower while the proof stage was 29.9% faster. Its one-time scout cost 183.3 seconds and 88,468 net-new tokens; charging it once reduced its aggregate active-time advantage from 22.5% to 19.9%. None of these figures can be directly subtracted from the fixed-target results: task corpora, models, clocks, proof obligations, source-line denominators, and token accounting differ. The ten-task corpus also has explicit outcome-aware selection provenance.¹⁰

⁹Independent report and source: NumStability ten_task_benchmark branch, Documentation/report.tex and benchmark/results/summary.json; PDF SHA-256 8b2cd10787487cb4..., summary SHA-256 02a5451409519335....

¹⁰NumStability benchmark/results/summary.json and Documentation/numstability_ten_task_report.pdf.

8 Interpretation, limitations, and future eligibility

The most defensible claim from this report is conditional and descriptive: *within this selected archive, tasks whose final L proofs consistently used NumStability often show larger time and submitted-code gains than tasks with no observed use*. The claim is compatible with useful reusable mathematics, but it does not prove that final declaration use caused the gain, nor that unused tasks had no useful library material. L could lose time on search, choose an incompatible interface, or fail to find a relevant declaration; the accepted-proof audit cannot distinguish those mechanisms on its own.

- **Campaign confounding.** The no-use/consistent-use contrast is highly aligned with r09/r10 versus r11. These campaigns differ in agent parallelism and provider context. Even a tier-free chart does not eliminate that problem.
- **Post-treatment grouping.** Reuse is observed after the agent has worked; it is not randomized. Easier proof surfaces may promote both library uptake and faster completion.
- **Inexact tokens.** All selected old-campaign token records are incomplete. Their values are provider-observed lower bounds, not exact total spending or directly comparable with the ten-task study’s net-new-token measure.
- **Task validity.** The fixed Lean statement, not a newly generated paper formalization, was the target. P12-T1/P17-T1 are superseded target versions, and P16-T3 was a known false-target failure outside this accepted-pair analysis. Future source-faithfulness or mathematical-scope screening may change the eligible corpus; it should be documented before recomputing results.
- **Usage semantics.** An audited transitive declaration is stronger evidence than an import, but may still be only background support. The candidate-list and semantic-result scores in the companion analysis offer context, yet depend partly on the original task design.

If the thesis narrows the intended numerical-stability scope, a subsequent report can predeclare eligibility from paper content and formalization compatibility, list every excluded task and reason, and retain this complete 38-task analysis as a sensitivity view. The better follow-up test of the proposed mechanism would keep one model/protocol/corpus and compare task-relevant library support with consistent search accounting, then inspect full conversations for retrieval costs and missed opportunities. This report is not a substitute for that prospective test.

9 Sources and reproducibility

The starting point is the companion `../library-usage-analysis/` folder on this branch. It contains `analyze.py`, `attempt_usage.csv` (accepted source/audit paths and SHA-256 values), `pair_usage.csv`, `task_usage.csv`, `declaration_usage.csv`, source-input hashes, and the original tiered usage report, which has not been edited. Source pair summaries are `../live-r11/completed_pairs.csv` and `../combined-r09-r10/completed_pairs.csv`; their READMEs describe metric definitions and campaign selection. Historical execution decisions are recorded in `../MEASUREMENT_PROGRESS.md`. The independent comparison source is the checked NumStability `ten_task_benchmark` branch’s `Documentation/` and `benchmark/results/summary.json`. The external report is cited by hash above because it is not copied into this branch.

From the repository root, regenerate this report’s tables and SVGs with `python3 paper_benchmark/highambench/reports/usage-without-tiers/make_figures.py`. From this report directory, render the four SVGs and compile:

```
sips -s format png figures/*.svg -out figures
latexmk -pdf -interaction=nonstopmode -halt-on-error report.tex
```

These commands regenerate only derived report assets; they do not rerun contestants or alter measured inputs. The current PDF is a snapshot of the cited hashes and dates.

Appendix: source-paper index

The table maps all 38 archived task IDs to their 31 paper identifiers. Titles and years are read from `../../metadata/manifest.json`; that current metadata provides bibliographic identity, not a claim that every historical target is still the present construction target. Exact paper locators and hashes reside in the manifest and individual task records.

Table 2: All analyzed task IDs and their paper sources.

Paper	Archived task IDs	Source title and year
P01	P01-T1, P01-T2	The Accuracy of Floating Point Summation (1993)
P02	P02-T1, P02-T3	Accurate Sum and Dot Product (2005)
P03	P03-T3	Accelerating the Solution of Linear Systems by Iterative Refinement in Three Precisions (2018)
P04	P04-T1	Mixed Precision Block Fused Multiply-Add: Error Analysis and Application to GPU Tensor Cores (2020)
P05	P05-T3	Improved Backward Error Bounds for LU and Cholesky Factorizations (2014)
P06	P06-T3	Probabilistic Rounding Error Analysis of Householder QR Factorization (2023)
P08	P08-T3	Iterative Refinement Implies Numerical Stability for Gaussian Elimination (1980)
P09	P09-T3	Roundoff Error Analysis of the Fast Fourier Transform (1971)
P11	P11-T3	A note on the error analysis of classical Gram-Schmidt (2006)
P12	P12-T1	A note on Dekker's FastTwoSum algorithm (2020)
P13	P13-T3	The numerical stability of barycentric Lagrange interpolation (2004)
P14	P14-T1	Accurately computing the log-sum-exp and softmax functions (2021)
P15	P15-T1, P15-T2, P15-T3	Solving block low-rank linear systems by LU factorization is numerically stable (2022)
P17	P17-T1, P17-T3	Probabilistic Error Analysis of Limited-Precision Stochastic Rounding (2025)
P19	P19-T3	Mixed precision strategies for preconditioned GMRES: a comprehensive analysis (2026)
P20	P20-T3	Error Analysis of Matrix Multiplication with Narrow Range Floating-Point Arithmetic (2025)
P23	P23-T1	Mixed-Precision Paterson-Stockmeyer Method for Evaluating Polynomials of Matrices (2025)
P24	P24-T3	Improving the Numerical Stability of Fast Matrix Multiplication (2016)
P25	P25-T3	A New Scaling and Squaring Algorithm for the Matrix Exponential (2009)
P26	P26-T1	Shifted Cholesky QR for Computing the QR Factorization of Ill-Conditioned Matrices (2020)
P27	P27-T3	Efficient Algorithms for Computing a Strong Rank-Revealing QR Factorization (1996)
P28	P28-T1, P28-T3	Backward Stability of Iterations for Computing the Polar Decomposition (2012)
P29	P29-T1, P29-T2	Accurate Symmetric Indefinite Linear Equation Solvers (1998)
P32	P32-T3	Backward Error and Condition of Structured Linear Systems (1992)
P33	P33-T1	Solving Sparse Linear Systems with Sparse Backward Error (1989)
P34	P34-T3	Accurate Computations with Totally Nonnegative Matrices (2007)
P35	P35-T3	Jacobi's Method is More Accurate than QR (1992)
P36	P36-T3	A Schur-Parlett Algorithm for Computing Matrix Functions (2003)
P37	P37-T3	Low Rank Solution of Lyapunov Equations (2002)
P39	P39-T3	Choosing the Forcing Terms in an Inexact Newton Method (1996)
P40	P40-T3	Computing the Nearest Correlation Matrix—A Problem from Finance (2002)